

WhatsApp SACCO AI Companion

Enterprise Dual-Sided AI Member Companion & Operations Engine for Kenyan SACCOs

SYSTEM 1 – 12 IMPLEMENTED

HANDOVER TARGET: GEMINI / ENGINEERING TEAM

Executive Mission: Kenyan Savings and Credit Cooperative Organizations (SACCOs) serve millions of members ranging from tech-savvy young professionals to rural, less digitally literate elders. Traditional customer service (long branch queues, congested phone hotlines, and complex online portals) creates severe friction. This platform provides an **intelligent, conversational WhatsApp-first member companion** paired with a **centralized multi-tenant staff operations dashboard**, delivering instant, grounded SACCO assistance with zero hallucinations and sub-second response times.

1. Core Value Proposition: "Not Just Another Chatbot"

The system rejects the traditional open-ended LLM chatbot model that frequently invents interest rates, dividends, or loan limits. Instead, it enforces strict deterministic separation between general knowledge RAG, mathematical planning, and sensitive personal financial data:

CARDINAL ARCHITECTURAL PRINCIPLE

Sensitive & member-specific requests must NEVER pass through general document RAG. When a member asks *"What is my loan balance?"* or *"How much can I borrow?"*, semantic vector search is strictly forbidden. The request routes to an authoritative **Member Data Service** fetching deterministic ledger records via SHA-256 phone hashing.

ZERO-HALLUCINATION MATH ENGINE

LLMs are strictly forbidden from performing financial arithmetic. All financial goal projections, required monthly savings, interest amortizations, and what-if scenarios are calculated deterministically by `GoalCalculator` in <1ms before presentation.

2. End-to-End Architectural Topology & Data Flow



3. Complete Subsystems Breakdown (Systems 1 – 12)

The project follows an evidence-driven progression across 12 distinct, fully tested modular subsystems:

SYSTEM	COMPONENT NAME	STATUS	CORE DELIVERABLES & ARCHITECTURAL SPECIFICATIONS
System 1	FastAPI / WhatsApp Core	PASS	FastAPI asynchronous web server, <code>/health</code> liveness probe, <code>/readiness</code> dependency probe, Twilio WhatsApp webhook receiver, and TwiML XML response serialization.
System 2	LLM Provider Abstraction	PASS	Provider-agnostic interface (<code>app/ai/llm.py</code>). Powered by Groq Cloud (<code>openai/gpt-oss-120b</code>) with strict 8,000 TPM rate limiting, fallback error envelopes, and swappable provider design.
System 3	Flexible Triage & Router	PASS	Eliminated rigid intent taxonomies. Candidate filtering and dynamic classification: <code>needs_member_data</code> , <code>likely_needs_human</code> , <code>language</code> (English, Kiswahili, Sheng).
System 4	RAG Vector Foundation	PASS	Qdrant vector engine adapter (Dockerized), deterministic 1000-char chunking with 200-char overlap, local Sentence Transformers (<code>all-MiniLM-L6-v2</code>), and <code>/rag/search</code> API.
System 5	Grounded Answer Generation	PASS	Grounding context builder, source attribution metadata citations, strict non-hallucination prompt boundaries, and standardized fallback taxonomy when evidence is insufficient.
System 6	RAG Hardening & Evaluation	PASS	Follow-up query reformulation, retrieval answerability gating (min score 0.20), PostgreSQL conversation history (20 turns bounded, 90-day retention), offline/live evaluation harness.
System 7	Member Identity & Fast-Path	PASS	SHA-256 phone hashing for member privacy. Deterministic fast-path bypassing LLM entirely for balance, account, and loan queries in <1 second with 0% hallucination risk.
System 8	Financial Goals Planning	PASS	Deterministic financial math calculator (<code>GoalCalculator</code>), natural language goal extractor, progress tracking, scenario what-if calculators, and WhatsApp response hygiene.
System 9	Personalized Coaching	PASS	Adaptive educational coaching based on member literacy (<code>beginner</code> , <code>intermediate</code> , <code>advanced</code>), active financial goals, and strict non-directive safety bounds (rejects crypto/stock tips).
System 10	Proactive Alerts & News	PASS	Goal milestone alerts, goal-matched verified financial news summaries, quiet-hours policy controller (20:00–08:00 EAT), 30-day deduplication, and two-way member feedback loops ("MORE", "ENDELEA").
System 11	SACCO Admin Operations	PASS	Multi-tenant staff authentication (PBKDF2 + JWT), automated human escalation ticket queue, knowledge document review/approval state machine with instant Qdrant vector sync, glassmorphic UI at <code>/dashboard</code> .
System 12	Core Banking & Auth Integration	PASS	SACCO Sandbox API adapter, session-based member authentication (PIN/OTP), tenant isolation guardrails, read-write separation for change requests, and full mock test coverage.

4. Comprehensive Technology Stack & Runtime Matrix

LAYER	TECHNOLOGY	VERSION / MODEL	IMPLEMENTATION DETAILS & ARCHITECTURAL RATIONALE
Language / Runtime	Python (Conda Env)	3.12.x	Isolated conda environment: whatsapp_sacco . Native asynchronous execution, strong typing with Pydantic.
API Framework	FastAPI + Uvicorn	0.115.6 / 0.34.0	High-performance async ASGI server with pre-warming lifespan routines to eliminate cold starts on webhooks.
WhatsApp Channel	Twilio Messaging API	9.3.5	Inbound webhook at POST /webhooks/whatsapp returning compliant TwiML messaging XML responses.
Tunneling	Cloudflare Quick Tunnel	cloudflared (win)	Zero-cost, secure HTTPS ingress tunnel mapping local port 8000 to Twilio Sandbox without third-party rate limits.
Relational Database	PostgreSQL 18	18-x64 (psycpg2)	Stores conversation history, SHA-256 member records, financial goals, news, preferences, and admin escalation tickets.
Vector Database	Qdrant (Docker)	v1.13.2	Vector store on ports 6333/6334 using cosine similarity and payload metadata filtering (sacco_id , language).
Embeddings Engine	Sentence Transformers	all-MiniLM-L6-v2	Local 384-dimensional dense neural embeddings. Provider-agnostic, zero network latency, zero per-query API costs.
Primary LLM	Groq Cloud API	openai/gpt-oss-120b	Ultra-fast token inference for RAG synthesis and coaching. Low reasoning effort. Budgeted under 8,000 TPM limit.
Staff Dashboard UI	HTML5 / ES6 / CSS3	Glassmorphic SPA	Mounted at /dashboard via FastAPI StaticFiles. Includes KPI cards, ticket queue, and policy sync.
Security & Crypto	PBKDF2 & SHA-256	hashlib / hmac	Deterministic SHA-256 phone hashing for member data privacy; PBKDF2 salt hashing for staff credentials.
Testing / Quality	Pytest + Custom Suite	372 Tests	Comprehensive unit, integration, and offline evaluation harness measuring Recall@K, Answerability, and Groundedness.

5. Verified Evaluation & Quality Metrics

The system was hardened through a strict scientific feedback loop across 84 test cases in data/evaluation/rag_eval.json :

RETRIEVAL & QUALITY METRICS		SAFETY, ROUTING & ROBUSTNESS	
Recall@1	68.18% (Target for hybrid BM25 optimization)	Hallucination Count	0 (Absolute zero tolerance)
Recall@3 & Recall@5	100.00% PASS (Zero missed policies)	High-Risk Hallucinations	0 (Zero invented rates or figures)
Answerability Accuracy	100.00% PASS (34/34 answerable queries)	Routing & Fallback Accuracy	100.00% (84/84 cases handled correctly)
Groundedness Score	100.00% PASS (Strict source alignment)	Wrong Routing Incidents	0 (Member data never leaks into RAG)
Relevance Score	100.00% PASS (Zero off-topic answers)	Passing Unit/Integration Tests	372 Tests Collected (Zero failures)

6. Developer & Gemini Handover Protocol (Operational Playbook)

Immediate Instructions for Gemini / Incoming Engineer:

- **Environment Activation:** Always activate conda env `whatsapp_sacco` (`conda activate whatsapp_sacco`).
- **Start Vector Store:** `docker compose -f docker_compose.yml up -d qdrant` (verifies collection `sacco_knowledge`).
- **Start Application Server:** `uvicorn app.main:app --reload --port 8000` (pre-warms embeddings and conversation services on startup).
- **Run Database Migrations:** `python scripts/migrate.py` (executes migrations 001 to 006 explicitly; app never auto-mutates schemas).
- **Run Test Suite:** `python -m pytest -v` (validates all 372 collected unit and integration tests).
- **Run Offline RAG Evaluation:** `python scripts/evaluate_rag.py` (tests retrieval and answerability without consuming Groq tokens).
- **Expose to Twilio:** Launch `cloudflared-windows-amd64.exe tunnel --url http://localhost:8000` and paste URL into Twilio Sandbox webhook (`/webhooks/whatsapp`).

KEY CODEBASE STRUCTURE

- `app/api/routes/` : FastAPI routers (whatsapp, rag, members, goals, education, proactive, admin, sacco_sandbox)
- `app/services/conversations/` : Context, state, triage pre-filter, dispatcher, workflow handlers
- `app/services/goals/` : Deterministic math calculator and natural language goal extractor
- `app/services/rag/` : Grounded answer service, knowledge gap intelligence, Qdrant repository
- `app/static/dashboard/` : Modern glassmorphic Admin Operations SPA

KEY ARCHITECTURAL CONSTRAINTS

1. **Do Not Re-introduce LLM Math:** Keep `GoalCalculator` pure Python.
2. **Never Pass Member Balances to RAG:** Enforce the member-data fast-path separation.
3. **Observe Groq 8k TPM Limit:** Avoid unconstrained parallel LLM chains.
4. **Preserve Privacy:** Never log or persist plain-text phone numbers; use SHA-256 hashes.

7. Roadmap & Expansion Priorities (Next Milestones)

MILESTONE	FOCUS AREA	DESCRIPTION & OBJECTIVES
Phase 13.1	Voice Messaging & Audio	Integrate OpenAI Whisper API to transcribe incoming WhatsApp voice notes into text for low-literacy members.
Phase 13.2	Receipt & Document OCR	Ingest and parse M-Pesa transaction confirmation SMS screenshots and PDF bank deposit slips sent over WhatsApp.
Phase 13.3	Live Core Banking Connectors	Transition from the sandbox mock adapter to live ISO 8583 / REST banking core webhooks with real SACCO partners.
Phase 13.4	Hybrid BM25 Retrieval	Implement BM25 lexical + dense vector hybrid search in Qdrant to boost Top-1 retrieval recall from 68.18% to >85%.